



Elliptic Curve Cryptosystems in the Presence of Permanent and Transient Faults

MATHIEU CIET*

ciet@dice.ucl.ac.be

Université Catholique de Louvain, UCL Crypto Group, Place du Levant 3, 1348 Louvain-la-Neuve, Belgium

MARC JOYE

marc.joye@gemplus.com

Gemplus, Card Security Group, La Vigie, Avenue du Jujubier, ZI Athélia IV, 13705 La Ciotat Cedex, France

Communicated by: P. Wild

Received December 11, 2002; Revised August 26, 2003; Accepted September 11, 2003

Abstract. Elliptic curve cryptosystems in the presence of faults were studied by [Biehl et al., *Advances in Cryptology – CRYPTO 2000*, Springer Verlag (2000) pp. 131–146]. The first fault model they consider requires that the input point P in the computation of dP is *chosen* by the adversary. Their second and third fault models only require the knowledge of P . But these two latter models are less ‘practical’ in the sense that they assume that only a *few* bits of error are inserted (typically exactly one bit is supposed to be disturbed) either into P just prior to the point multiplication or during the course of the computation in a chosen location.

This paper relaxes these assumptions and shows how *random* (and thus unknown) errors in either coordinates of point P , in the elliptic curve parameters or in the field representation enable the (partial) recovery of multiplier d . Then, from multiple point multiplications, we explain how this can be turned into a total key recovery. Simple precautions to prevent the leakage of secrets are also discussed.

Keywords: elliptic curve cryptography, fault analysis, fault attacks, information leakage

1. Introduction

Elliptic curve cryptography was introduced in the mid 1980s independently by Koblitz [14] and Miller [21] as a promising alternative for cryptographic protocols based on the discrete logarithm problem in the multiplicative group of a finite field (e.g., Diffie-Hellman key exchange or ElGamal encryption/signature). The security of elliptic curve cryptosystems relies on the hardness of solving the elliptic curve discrete logarithm problem (ECDLP): given points P and $Q = dP$ on an elliptic curve, one has to recover multiplier d .

*The work described in this paper has been supported [in part] by the Commission of the European Communities through the IST Programme under Contract IST-1999-12324, <http://www.cryptones-sie.org/>. The information in this document is provided as is, and no guarantee or warranty is given or implied that the information is fit for any particular purpose. The user thereof uses the information at his sole risk and liability. The views expressed are those of the authors and do not represent an official view/position of the NESSIE project (as a whole).

1.1. Physical Security

Kocher et al. introduced the notion of *side-channel analysis* in [15, 16] and showed the importance for an implementation of being resistant against side-channel analysis (e.g., secret leakage from power consumption). Resistance against fault analysis [7, 5] is another threat and should be taken into account, since sensitive information may leak when the cryptosystem operates under unexpected conditions.

The security of point multiplication on elliptic curves in the presence of faults was considered by Biehl et al. [4]. They extended fault attacks initially mounted against the RSA cryptosystem to schemes using elliptic curves. Remarking that elliptic curve parameter a_6 is not used in the classical addition formulæ on elliptic curves, they made the key observation that a random element $(\tilde{x}, \tilde{y})$ (not on the original elliptic curve) may define a group for which it is computationally feasible to solve the ECDLP. Therefore, by forcing the input point as $\tilde{P} = (\tilde{x}, \tilde{y})$, they were able to derive information about d from the output value $d\tilde{P}$.

Many elliptic curve cryptosystems do not allow to choose (or force) the input point. In the second part of their paper, Biehl et al. suppose that an error of one bit occurs and then try to recover it by successive search.

1.2. Our Contribution

In this paper, we rather consider a more practical scenario and give several refinements of [4].

Typically, in a cryptographic device, the system parameters are stored in non-volatile memory (e.g., EEPROM) and are then transferred into working memory (e.g., RAM), when needed, for the elliptic curve computations. In our first model, we assume that there is a *permanent fault*, in a *unknown* position, in *any* system parameter defining the elliptic curve over which the computations are carried out. In our second model, we analyze the consequences of faults during the transfer of the system parameters into working memory. We call such a fault a *transient fault*. In both models, we explain how this may yield information on (secret) multiplier d .

The rest of this paper is organized as follows. In the next section, we review the basics of elliptic curve cryptography. Then, in Section 3, we exhibit how the presence of faults in the public parameters of an elliptic curve cryptosystem may expose the secret key. Finally, we conclude in Section 4.

2. Elliptic Curve Cryptography

An elliptic curve over a field $\mathbb{K}$ is the set of points $(x, y) \in \mathbb{K} \times \mathbb{K}$ satisfying the Weierstraß equation

$$E/\mathbb{K} : y^2 + a_1 xy + a_3 y = x^3 + a_2 x^2 + a_4 x + a_6 \quad (1)$$

along with the point $\mathcal{O}$ at infinity. This set of points form an *additive group* where $\mathcal{O}$ is the neutral element and where the group law is given by the

‘chord-and-tangent’ rule. The inverse of a point $P_1 = (x_1, y_1)$ is $-P_1 = (x_1, -y_1 - a_1 x_1 - a_3)$. Given two points $P_1 = (x_1, y_1)$ and $P_2 = (x_2, y_2)$ (with $P_1 \neq -P_2$), the sum $P_3 = P_1 + P_2 = (x_3, y_3)$ is defined as

$$x_3 = \lambda^2 + a_1 \lambda - a_2 - x_1 - x_2 \quad \text{and} \quad y_3 = (x_1 - x_3)\lambda - y_1 - a_1 x_3 - a_3 \quad (2)$$

$$\text{with } \lambda = \begin{cases} \frac{3x_1^2 + 2a_2x_1 + a_4 - a_1y_1}{2y_1 + a_1x_1 + a_3} & \text{if } P_1 = P_2, \\ \frac{y_1 - y_2}{x_1 - x_2} & \text{otherwise.} \end{cases}$$

Remark. When the characteristic of $\mathbb{K}$, $\text{Char } \mathbb{K}$, is a ‘large’ prime (namely, ≥ 5), one can take $a_1 = a_2 = a_3 = 0$ in the Weierstraß parameterization, without loss of generality. Likewise when $\text{Char } \mathbb{K} = 2$, one can take $a_1 = 1$ and $a_3 = a_4 = 0$, provided that the elliptic curve is non-supersingular. This is the case of interest for cryptographic applications since supersingular curves are prone to the mov attack [19].

The scalar multiplication (or point multiplication) is the operation consisting in computing $Q = dP = P + P + \dots + P$ (d times). A related operation is the *discrete logarithm*, which consists in computing d from P and $Q = dP$, and is the basis for the security of cryptographic schemes based on elliptic curves over finite fields [14, 21]. The discrete logarithm problem on a *general* elliptic curve (ECDLP) is particularly attractive since there is no known sub-exponential algorithm to solve it. The best method available for attacking the ECDLP is Pollard’s method [22, 23] and variants thereof.

3. Permanent and Transient Faults

Consider the elliptic curve $E(a_1, a_2, a_3, a_4, a_6)$ over $\mathbb{K}$ given by equation (1). In [4], Biehl et al. observe that parameter a_6 is not involved in the addition formulae (cf. equation (2)). Consequently, if a cryptographic device (e.g., a smart card) receives on input a ‘point’ $\tilde{P} = (\tilde{x}, \tilde{y}) \in \mathbb{K} \times \mathbb{K}$ but $\tilde{P} \notin E$ then the scalar multiplication $d\tilde{P}$ will take place over the curve $\tilde{E}(a_1, a_2, a_3, a_4, \tilde{a}_6)$ with

$$\tilde{a}_6 = \tilde{y}^2 + a_1 \tilde{x} \tilde{y} + a_3 \tilde{y} - \tilde{x}^3 - a_2 \tilde{x}^2 - a_4 \tilde{x}$$

instead of over the original curve $E(a_1, a_2, a_3, a_4, a_6)$.

Assume that point $\tilde{P}$ is chosen so that $\tilde{E}(a_1, a_2, a_3, a_4, \tilde{a}_6)$ is an elliptic curve whose order has a small (or smooth) factor r and that the order of $\tilde{P}$ as an element of $\tilde{E}$, $\text{ord}_{\tilde{E}}(\tilde{P})$, is equal to r . Then, provided that discrete logarithms are computable in $\langle \tilde{P} \rangle$, the subgroup of order r generated by $\tilde{P}$, the value of $d \pmod{r}$ can be recovered.¹ Therefore, repeating the process with sufficiently many different *chosen* points $\tilde{P}_i$ yields the values of $d \pmod{r_i}$ (where $r_i = \text{ord}_{\tilde{E}_i}(\tilde{P}_i)$) from $d\tilde{P}_i$, and so the whole value of d , by Chinese remaindering.

Most elliptic curve cryptosystems take a *determined* point P on input, making the previous attack inapplicable (a noteworthy exception is the basic ElGamal

encryption [9]). Indeed, being a system parameter, point $\mathbf{P}$ is stored in the non-volatile memory of the cryptographic device and is read from that memory for the computation of $d\mathbf{P}$.

In this section, we analyze the implications of *permanent* faults (intentional or accidental) in non-volatile memory, where the system parameters are stored. We alternately consider permanent faults in the representation of point $\mathbf{P}$, in the definition field $\mathbb{K}$, and in curve parameters $\{a_1, \dots, a_6\}$. Our analysis also extends to *transient* faults originating from a perturbation of the reading in non-volatile memory, and resulting in faulty values for the system parameters actually used in working memory, throughout the computation. The difference between the two models is that a transient fault, as defined, is fixed throughout a *single* execution (but may vary from one execution to another one) whereas a permanent fault always sticks invariant.

3.1. Faults in the Base Point

Without loss of generality, we assume that the x -coordinate of point $\mathbf{P}$ is corrupted (the case of a corrupted y -coordinate is similar). The cryptographic device then computes $\tilde{\mathbf{Q}} = d\tilde{\mathbf{P}}$ with $\tilde{\mathbf{P}} = (\tilde{x}, y)$. Contrary to the case considered in [4], the value of $\tilde{\mathbf{P}}$ is *unknown* but fixed (in particular, there is no hypothesis on the number of flipped bits). It is however easy to recover the value of $\tilde{\mathbf{P}}$ from the output value $\tilde{\mathbf{Q}} = d(\tilde{x}, y) = (\tilde{x}_d, \tilde{y}_d)$. Point $\tilde{\mathbf{Q}}$ defines the curve $\tilde{E}$ on which the computation was carried out, i.e., $\tilde{E}(a_1, a_2, a_3, a_4, \tilde{a}_6)$ with

$$\tilde{a}_6 = \tilde{y}_d^2 + a_1 \tilde{x}_d \tilde{y}_d + a_3 \tilde{y}_d - \tilde{x}_d^3 - a_2 \tilde{x}_d^2 - a_4 \tilde{x}_d. \quad (3)$$

Therefore, since $\tilde{\mathbf{P}} = (\tilde{x}, y) \in \tilde{E}$, it follows that $\tilde{x}$ is a root in $\mathbb{K}$ of the polynomial (over $\mathbb{K}[X]$) given by

$$X^3 + a_2 X^2 + (a_4 - a_1 y)X + (\tilde{a}_6 - y^2 - a_3 y).$$

Suppose first that this polynomial has a unique root, which must be $\tilde{x}$. Provided that $r = \text{ord}_{\tilde{E}}(\tilde{\mathbf{P}})$ (i.e., the order of $\tilde{\mathbf{P}} = (\tilde{x}, y)$ in $\tilde{E}$) is small (or smooth) enough so that the discrete logarithm of $\tilde{\mathbf{Q}}$ w.r.t. $\tilde{\mathbf{P}}$ is computable, the value of $d \bmod r$ can be recovered.

If now the above polynomial has 2 or 3 roots, then there are 2 or 3 possible candidates for $\tilde{x}$. In the permanent-fault model, the ambiguity can however be removed by assuming that only a portion of x is corrupted (remember that parameter $\mathbf{P} = (x, y)$ is public). Hence, the candidate having the most bits matching those of x is likely to be $\tilde{x}$. In the transient-fault model, the whole value of x is likely to be corrupted; the actual value of $\tilde{x}$ is then known with probability 1, $\frac{1}{2}$ or $\frac{1}{3}$, according to the number of candidates. Next, once $\tilde{\mathbf{P}} = (\tilde{x}, y)$ is known (or more exactly, guessed), the value of $d \bmod r$ can be recovered.

Remark. If it is the y -coordinate of $\mathbf{P}$ that is corrupted (i.e., $\tilde{\mathbf{P}} = (x, \tilde{y})$) then $\tilde{y}$ is a root of a polynomial of degree 2 and the above methodology applies to recover

the value of $d \bmod r$. (Note that an error in the y -coordinate only makes sense when there is no point compression² —with point compression, an error merely changes $\mathbf{P}$ into $-\mathbf{P}$.)

A more intricate case appears when both the x - and the y -coordinates of $\mathbf{P}$ are corrupted. We write $\tilde{\mathbf{P}} = (\tilde{x}, \tilde{y})$ the corresponding point. As before, the output value $d\tilde{\mathbf{P}} = (\tilde{x}_d, \tilde{y}_d)$ yields the value of $\tilde{a}_6$ from equation (3). But it seems that there is no way to recover the whole value of $\tilde{\mathbf{P}}$; we only know that it lies on the curve $\tilde{E}(a_1, a_2, a_3, a_4, \tilde{a}_6)$. Further assumptions are needed to completely recover $\tilde{\mathbf{P}}$. For example, in case of permanent fault, if it is possible to flip one bit of d , say bit d_i , during the computation of $d\tilde{\mathbf{P}}$ then the value of $\pm 2^i \tilde{\mathbf{P}}$ can be obtained as $d\tilde{\mathbf{P}} - d'\tilde{\mathbf{P}}$ where

$$d' = d + (\neg d_i - d_i) 2^i.$$

Successively halving $\pm 2^i \tilde{\mathbf{P}}$ on $\tilde{E}(a_1, a_2, a_3, a_4, \tilde{a}_6)$ eventually yields the value of $\pm \tilde{\mathbf{P}}$. Then, once $\pm \tilde{\mathbf{P}}$ is retrieved, the value of $\pm d \bmod r$ can be recovered. This extends the attacks of [3, 13] in the sense that more information on secret d (i.e., $\pm d \bmod r$) can be recovered.

3.2. Faults in the Definition Field

We now suppose that the representation of field $\mathbb{K}$ in non-volatile memory is faulty (permanent fault) or that an error occurred in the transfer from non-volatile memory to working memory (transient fault). We have to handle the cases $\mathbb{K} = \mathbb{F}_p$ and $\mathbb{K} = \mathbb{F}_{2^q}$ separately since their internal representations are different.

3.2.1. Large Prime Field

Let p be a large prime number. The field $\mathbb{F}_p$ is simply represented by the value of p , stored as a binary string in non-volatile memory. An error on $\mathbb{K}$ means in this case an error in the representation of p . We let $\tilde{p}$ denote the faulty value. The computations are then carried out modulo $\tilde{p}$ instead of modulo p .

As mentioned in Section 2, elliptic curves over a large prime field are given by the simplified equation

$$E/\mathbb{F}_p : y^2 = x^3 + a_4 x + a_6.$$

Over $\mathbb{F}_{\tilde{p}}$, point $\mathbf{P} = (x, y)$ becomes $\tilde{\mathbf{P}} = (\tilde{x}, \tilde{y})$ where $\tilde{x} \equiv x \pmod{\tilde{p}}$ and $\tilde{y} \equiv y \pmod{\tilde{p}}$. The computation of $d\mathbf{P}$ on E is replaced by $d\tilde{\mathbf{P}}$ on the curve $\tilde{E}$ defined by³

$$\tilde{E}/\mathbb{F}_{\tilde{p}} : y^2 = x^3 + \tilde{a}_4 x + \tilde{a}_6$$

where $\tilde{a}_4 \equiv a_4 \pmod{\tilde{p}}$. Since both $\tilde{\mathbf{P}} = (\tilde{x}, \tilde{y})$ and $\tilde{\mathbf{Q}} = d\tilde{\mathbf{P}} = (\tilde{x}_d, \tilde{y}_d)$ are points on $\tilde{E}$, we deduce that

$$\tilde{a}_6 \equiv \tilde{y}^2 - \tilde{x}^3 - \tilde{a}_4 \tilde{x} \equiv y^2 - x^3 - a_4 x \equiv \tilde{y}_d^2 - \tilde{x}_d^3 - a_4 \tilde{x}_d \pmod{\tilde{p}}. \quad (4)$$

Letting $R = y^2 - x^3 - a_4 x$ and $\tilde{R}_d = \tilde{y}_d^2 - \tilde{x}_d^3 - a_4 \tilde{x}_d$ (R and $\tilde{R}_d$ are evaluated over $\mathbb{Z}$), the previous relation implies that $\tilde{p}$ divides $\Delta = R - \tilde{R}_d$. Consequently, the factorization of Δ will reveal the value of $\tilde{p}$ as the combination of the factors of Δ whose product has the most bits matching those of p . If no such $\tilde{p}$ is found (e.g., a transient fault gave rise to a random value for $\tilde{p}$), we have to assume that the original prime, p , satisfies additional properties. A concrete example is discussed in § 3.2.3.

Remark. Although being a $3k$ -bit integer (where k is the bit-length of prime p), Δ is within the range of modern factorization algorithms since k is typically a 160-bit integer. Furthermore, when the fault is permanent, the length of the number to be factored can be lowered from an additional scalar multiplication with point $\tilde{P}$. Let $\tilde{Q}' = d' \tilde{P} = (\tilde{x}_{d'}, \tilde{y}_{d'})$ with $d' \neq d$. Then $(\tilde{x}_{d'}, \tilde{y}_{d'})$ satisfies equation (4) and, letting $\tilde{R}_{d'} = \tilde{y}_{d'}^2 - \tilde{x}_{d'}^3 - a_4 \tilde{x}_{d'}$ and $\Delta' = R - \tilde{R}_{d'}$, it follows that $\tilde{p}$ must divide $\gcd(\Delta, \Delta')$. If further (distinct) scalar multiplications are available, $\tilde{p}$ can be obtained easilier as a factor of $\gcd(\Delta, \Delta', \Delta'', \dots)$.

We now assume that the value of $\tilde{p}$ is known. Let $\tilde{p} = \prod_{i=1}^l \tilde{q}_i^{e_i}$ denote the prime factorization of $\tilde{p}$. Consider the direct product of groups

$$\mathbb{G} = \tilde{E}_1(\mathbb{Z}/\tilde{q}_1^{e_1}\mathbb{Z}) \times \cdots \times \tilde{E}_l(\mathbb{Z}/\tilde{q}_l^{e_l}\mathbb{Z}).$$

Since the computation of $\tilde{Q} = k\tilde{P}$ was supposed successful, point $\tilde{Q}$ corresponds to a unique element of $\mathbb{G}$, namely $(\tilde{Q}_1, \dots, \tilde{Q}_l)$ with $\tilde{Q}_i = \tilde{Q} \bmod \tilde{q}_i^{e_i}$ [18, § 2.5]. Hence, if $r_i = \text{ord}_{\tilde{E}_i}(\tilde{P})$ denotes the order of $\tilde{P}$ viewed as an element of the elliptic curve $\tilde{E}_i$ (i.e., $\tilde{P}_i = \tilde{P} \bmod \tilde{q}_i^{e_i}$), then solving the discrete logarithm in $\langle \tilde{P}_i \rangle \subseteq \tilde{E}_i$ yields the value of $d \bmod r_i$ so that by Chinese remaindering the value of

$$d \bmod \text{lcm}(r_1, \dots, r_l)$$

can be calculated.

3.2.2. Binary Field

The field $\mathbb{F}_{2^q}$ is usually regarded as a quotient $\mathbb{F}_2[X]/(\mathcal{P})$ where $\mathcal{P}$ is an irreducible polynomial of degree q over $\mathbb{F}_2$. Each element $\alpha \in \mathbb{F}_{2^q}$ is represented as a binary string $(\alpha_0 \dots \alpha_{q-1})$ corresponding to polynomial $\sum_{i=0}^{q-1} \alpha_i X^i \pmod{\mathcal{P}(X)}$ in $\mathbb{F}_2[X]$. With this polynomial representation, the field $\mathbb{F}_{2^q}$ is determined by polynomial $\mathcal{P}$, which is stored in non-volatile memory as a binary string $(b_0 \dots b_{q-1} b_q)$ corresponding to $\mathcal{P}(X) = \sum_{i=0}^q b_i X^i$ with $b_q = 1$. Over $\mathbb{F}_{2^q}$, a (non-supersingular) elliptic curve is given by the simplified Weierstraß equation

$$E/\mathbb{F}_{2^q} : y^2 + xy = x^3 + a_2 x^2 + a_6.$$

Remark. There exist other representations for elements of $\mathbb{F}_{2^q}$. A common choice is the normal basis representation. We shall not consider this representation here,

as optimized implementations of elliptic curve doubling with normal bases make use of parameter a_6 [1, §. A.10.2]. Our analysis, however, implies that parameter a_6 is not used. Moreover, we assume that affine coordinates are used, as a projective doubling needs the value of a_6 [1, §. A.10.6]. Furthermore, note that affine coordinates lead to faster arithmetic than projective coordinates [8] in binary fields with polynomial representation.

Suppose that $\mathbb{K} = \mathbb{F}_{2^q}$ is faulty, namely that there is an error in the representation of $\mathcal{P}$. As a result, computations are performed modulo $\tilde{\mathcal{P}}(X) = \sum_{i=0}^q \tilde{b}_i X^i$ instead of modulo $\mathcal{P}(X)$. Viewing elements as polynomials over $\mathbb{K}[X]$, similarly to prime case, polynomial $\tilde{\mathcal{P}}$ can be recovered by observing that

$$\tilde{a}_6 \equiv y^2 + xy + x^3 + a_2 x^2 \equiv \tilde{y}_d^2 + \tilde{x}_d \tilde{y}_d + \tilde{x}_d^3 + a_2 \tilde{x}_d^2 \pmod{\tilde{\mathcal{P}}(X)}$$

where $(\tilde{x}_d, \tilde{y}_d) = d\tilde{\mathbf{P}}$ and $\tilde{\mathbf{P}} \equiv \mathbf{P} \pmod{\tilde{\mathcal{P}}(X)}$. Hence, letting

$$\Delta(X) = y^2 + xy + x^3 + a_2 x^2 + \tilde{y}_d^2 + \tilde{x}_d \tilde{y}_d + \tilde{x}_d^3 + a_2 \tilde{x}_d^2 \pmod{\mathbb{F}_2[X]}, \quad (5)$$

it follows that $\tilde{\mathcal{P}}(X)$ divides $\Delta(X)$. So, given the factorization of Δ , trying all possible combinations yields $\tilde{\mathcal{P}}$ as the polynomial whose representation best matches the representation of $\mathcal{P}$. Further, in case of permanent fault, an additional scalar multiplication, $\tilde{\mathbf{Q}}' = d'\tilde{\mathbf{P}}$ with $d' \neq d$, eases the recovery of $\tilde{\mathcal{P}}$ as a factor of $\gcd(\Delta, \Delta')$ where Δ' is defined from $\tilde{\mathbf{Q}}'$.

When the fault is transient, it is always possible to distinguish $\tilde{\mathcal{P}}$. In § 3.2.3, we present a technique that can successfully recover $\tilde{\mathcal{P}}$ in most practical implementations.

If polynomial $\mathcal{P}(X) = \sum_{i=0}^q b_i X^i$ representing the field $\mathbb{F}_{2^q}$ is modified into polynomial $\tilde{\mathcal{P}}(X) = \sum_{i=0}^q \tilde{b}_i X^i$, it is very probable that $\tilde{\mathcal{P}}$ is no longer irreducible over $\mathbb{F}_2$. Hence, we can write $\tilde{\mathcal{P}}$ as the product

$$\tilde{\mathcal{P}} = \tilde{\mathcal{P}}_1^{e_1} \dots \tilde{\mathcal{P}}_l^{e_l}$$

where $\tilde{\mathcal{P}}_i$ are distinct irreducible polynomials in $\mathbb{F}_2[X]$ and e_i are positive integers.

From the structure of the class residue ring $\mathbb{F}_2[X]/(\tilde{\mathcal{P}})$, we can collect information modulo the irreducible factors $\tilde{\mathcal{P}}_i$. Let $\tilde{\mathbf{P}}_i$ (resp. $\tilde{\mathbf{Q}}_i$) be the representative of $\tilde{\mathbf{P}}$ (resp. $\tilde{\mathbf{Q}}$) viewed as an element in the field $\mathbb{F}_{2^{\tilde{q}_i}} \cong \mathbb{F}_2[X]/(\tilde{\mathcal{P}}_i)$ where $\tilde{q}_i = \deg(\tilde{\mathcal{P}}_i)$, i.e., $\tilde{\mathbf{P}}_i = \tilde{\mathbf{P}} \pmod{\tilde{\mathcal{P}}_i}$ (resp. $\tilde{\mathbf{Q}}_i = \tilde{\mathbf{Q}} \pmod{\tilde{\mathcal{P}}_i}$). Therefore, the discrete logarithm of $\tilde{\mathbf{Q}}_i$ w.r.t. $\tilde{\mathbf{P}}_i$ in the group $\langle \tilde{\mathbf{P}}_i \rangle \subseteq \tilde{E}_i(\mathbb{F}_{2^{\tilde{q}_i}})$ is $d \pmod{r_i}$, where $r_i = \text{ord}_{\tilde{E}_i}(\tilde{\mathbf{P}}_i)$ and $\tilde{E}_i = \tilde{E}_i(\tilde{a}_{2,i}, \tilde{a}_{6,i})$ with

$$\tilde{a}_{2,i} \equiv a_2 \pmod{\tilde{\mathcal{P}}_i(X)} \text{ and } \tilde{a}_{6,i} \equiv y^2 + xy + x^3 + a_2 x^2 \pmod{\tilde{\mathcal{P}}_i(X)}.$$

Hence, Chinese remaindering on each subgroup $\tilde{E}_i$ yields the value of

$$d \pmod{\text{lcm}(r_1, \dots, r_l)}.$$

Remark that, when $\tilde{q}_i$ is composite, the computation of discrete logarithms in $\tilde{E}_i(\mathbb{F}_{2^{\tilde{q}_i}})$ can be speeded up by Weil descent [11] (see also [17, 20, 24] for a thorough analysis and [10, 12] for recent developments).

3.2.3. Unknown Error

The above analysis implies that the error on the representation of $\tilde{\mathbb{K}}$ is known. When only a portion of the representation of $\mathbb{K}$ is damaged, we choose for $\tilde{\mathbb{K}}$ the candidate best matching $\mathbb{K}$. In some cases, the error on (the representation of) $\mathbb{K}$ is completely random so that it is no longer possible to determine the resulting $\tilde{\mathbb{K}}$. We can merely restrict $\tilde{\mathbb{K}}$ as a set obtained by combining several known prime factors (case $\mathbb{K} = \mathbb{F}_p$) or irreducible polynomials (case $\mathbb{K} = \mathbb{F}_{2^q}$).

In this paragraph, we take advantage of some particularities of elliptic curves recommended for practical implementations to recover the right combination leading to $\tilde{\mathbb{K}}$ when the ‘best-matching’ strategy does not apply.

Over prime fields, the security of elliptic curve cryptography does not depend on the form of the primes. For example, in order to optimize the efficiency of field arithmetic, the elliptic curves recommended by the NIST [2]⁴ are defined over $\mathbb{F}_p$ where p is a (generalized) Mersenne prime, that is, a prime of the form

$$p = 2^{\omega_0} - \sum_{i=1}^B \pm 2^{\omega_i}$$

where B is chosen small [25]. Hence, if p is a Mersenne prime with $B \leq 4$ (this includes all NIST curves), it can be stored much more economically as $\{\omega_0, \sigma_1 \parallel \omega_1, \sigma_2 \parallel \omega_2, \sigma_3 \parallel \omega_3, \sigma_4\}$ and reconstructed in working memory as

$$p = 2^{\omega_0} + \sum_{i=1}^3 \sigma_i 2^{\omega_i} + \sigma_4 \quad (6)$$

with $\sigma_1, \sigma_2, \sigma_3 \in \{-1, 0, 1\}$ and $\sigma_4 \in \{-1, 1\}$. In this case, an error on p means an error on ω_0 , σ_4 and/or $\sigma_i \parallel \omega_i$ for some $1 \leq i \leq 3$. Then, instead of testing the combinations of factors of Δ having the most bits matching those of p (cf. § 3.2.1), we can conclude that the faulty $\tilde{p}$ is a number obtained as a combination of the factors of Δ , whose product can be written as a Mersenne number (non necessarily prime) satisfying equation (6). This additional assumption drastically reduces the number of candidates for $\tilde{p}$ and may allow to recover the value of $\tilde{p}$ even if the error is random (i.e., unknown).

A similar assumption holds for binary fields. For every q up to 1000, there exists an irreducible pentanomial $\mathcal{P}(X) = X^q + X^{q_1} + X^{q_2} + X^{q_3} + 1$ to represent the elements of $\mathbb{F}_{2^q}$ as polynomials in $\mathbb{F}_2[X]/(\mathcal{P})$ [1, § A.8]. Since the reduction of polynomials modulo a pentanomial is efficient, we may assume that only pentanomials are accepted for representing $\mathbb{F}_{2^q}$ (again, this includes all NIST curves⁵) and that they are stored in a compact way as $\{q, q_1, q_2, q_3\}$. An error on $\mathcal{P}$ translates in an error on q , q_1 , q_2 and/or q_3 , which can be easilier recovered by testing the factors of Δ (cf. § 3.2.2) that combine into a pentanomial $\tilde{\mathcal{P}}(X) = X^{\tilde{q}} + X^{\tilde{q}_1} + X^{\tilde{q}_2} + X^{\tilde{q}_3} + 1$.

3.3. Faults in the Curve Parameters

As parameter a_6 is not needed in the addition formulæ, a modification of its value does not affect the computation of $d\mathbf{P}$. So we only consider the situation of an error occurring in curve parameters a_1, a_2, a_3 or a_4 .

To fix the ideas, we assume that parameter a_4 is faulty (the other cases are similar). We let $\tilde{a}_4$ denote the corrupted value. As previously, since a_6 is not employed in the addition formulae, computations are performed on the curve $\tilde{E}(a_1, a_2, a_3, \tilde{a}_4, \tilde{a}_6)$. Furthermore, since both $\mathbf{P} = (x, y)$ and $\tilde{\mathbf{Q}} = d\mathbf{P} = (\tilde{x}_d, \tilde{y}_d)$ lie on the curve $\tilde{E}$, we have the system of equations

$$\begin{cases} \tilde{a}_4 x + \tilde{a}_6 = y^2 + a_1 xy + a_3 y - x^3 - a_2 x^2 \\ \tilde{a}_4 \tilde{x}_d + \tilde{a}_6 = \tilde{y}_d^2 + a_1 \tilde{x}_d \tilde{y}_d + a_3 \tilde{y}_d - \tilde{x}_d^3 - a_2 \tilde{x}_d^2 \end{cases}$$

whose resolution (over $\mathbb{K}$) gives the values of $\tilde{a}_4$ and $\tilde{a}_6$.

Remark. In the (very) unlikely event where the two equations are linearly dependent, the process can be re-iterated with another scalar multiplication.

After resolving the system of equations for $\tilde{a}_4$ and $\tilde{a}_6$, we compute the logarithm of $\tilde{\mathbf{Q}}$ w.r.t. $\mathbf{P}$ in $\langle \mathbf{P} \rangle \subseteq \tilde{E}(a_1, a_2, a_3, \tilde{a}_4, \tilde{a}_6)$ and get the value of $d \bmod r$, where $r = \text{ord}_{\tilde{E}}(\mathbf{P})$.

4. Concluding Remarks

We have shown that a (permanent) fault in the system parameters may enable one to recover the value of $d \bmod r$. However, it is fairly easy to avoid the leakage of $d \bmod r$ by checking the parameters for faults prior to the computation of $\mathbf{Q} = d\mathbf{P}$. This can be done by adding a CRC to each system parameter. Then, after reading a system parameter in non-volatile memory, its CRC is computed and compared with the CRC stored in non-volatile memory. Another possibility is to use curve parameter a_6 as an integrity check (i.e., a_6 is used to verify whether the coordinates of point $\mathbf{P} = (x, y)$ satisfy, over $\mathbb{K}$, the relation $y^2 + a_1 xy + a_3 y - x^3 - a_2 x^2 - a_4 x = a_6$).

To prevent the leakage of $d \bmod r$ in the presence of a transient fault, we have shown that the system parameters actually used (i.e., in working memory) must also be checked during the computation of $\mathbf{Q} = d\mathbf{P}$. In addition, we recommend to perform a check on point $\mathbf{Q}$ just before outputting it.

More importantly, our analysis teaches that not only the secret parameters (e.g., a secret key) but also *the public parameters must be checked for faults*.

Acknowledgments

Part of this work was done while the first author was visiting Gemplus. Thanks go to David Naccache, Philippe Proust and Jean-Jacques Quisquater for making this arrangement possible.

Notes

1. Note that r needs not be prime—when r is a smooth composite number, discrete logarithms can be attacked with the Pohlig–Hellman algorithm.
2. If the x -coordinate of point P is fixed, its y -coordinate satisfies a quadratic in y . Since a single bit is sufficient to distinguish between the two possible solutions of a quadratic, a point can be compressed into its x -coordinate and this additional bit.
3. When $\tilde{p}$ is not prime, $\mathbb{Z}_{\tilde{p}} = \mathbb{Z}/\tilde{p}\mathbb{Z}$ is no longer a field but a ring. As a result, the computation of $d\tilde{P}$ on $\tilde{E}$ is not guaranteed to succeed. This occurs when the computation involves a point that is the point at infinity modulo a factor $\tilde{p}_1$ of $\tilde{p}$ but not modulo $\tilde{p}_2 = \tilde{p}/\tilde{p}_1$.
4. The 5 NIST elliptic curves over large prime fields are defined with the Mersenne primes $2^{192} - 2^{64} - 1$ (curve P-192), $2^{224} - 2^{96} + 1$ (curve P-224), $2^{256} - 2^{224} + 2^{192} + 2^{96} - 1$ (curve P-256), $2^{384} - 2^{128} - 2^{96} + 2^{32} - 1$ (curve P-384) and $2^{521} - 1$ (curve P-521) [2].
5. The 10 NIST elliptic curves over binary fields are defined with the irreducible polynomials $X^{163} + X^7 + X^6 + X^3 + 1$ (curves K- and B-163), $X^{233} + X^{74} + 1$ (curves K- and B-233), $X^{283} + X^{12} + X^7 + X^5 + 1$ (curves K- and B-283), $X^{409} + X^{87} + 1$ (curves K- and B-409) and $X^{571} + X^{10} + X^5 + X^2 + 1$ (curves K- and B-571) [2].

References

1. IEEE Std 1363-2000. *IEEE Standard Specifications for Public-Key Cryptography*. IEEE Computer Society, August 29, 2000.
2. Federal Information Processing Standards Publication FIPS 186-2. *Digital Signature Standard (DSS)*, appendix 6: “Recommended elliptic curves for federal government use”. National Institute of Standards and Technology, January 27, 2000. Available at URL <http://csrc.nist.gov/publications/fips/fips186-2/fips186-2.pdf>.
3. F. Bao, R. H. Deng, Y. Han, A. B. Jeng, A. D. Narasimbalu and T.-H. Ngair, Breaking public key cryptosystems on tamper resistant devices in the presence of transient faults. In B. Christianson, B. Crispo, M. Lomas and M. Roe (eds), *Security Protocols*, Volume 1361 of *Lecture Notes in Computer Science*, Springer-Verlag (1997) pp. 115–124.
4. I. Biehl, B. Meyer and V. Müller. Differential fault attacks on elliptic curve cryptosystems. In M. Bellare (ed.), *Advances in Cryptology – CRYPTO 2000*, Volume 1880 of *Lecture Notes in Computer Science*, Springer-Verlag (2000) pp. 131–146.
5. E. Biham and A. Shamir, Differential fault analysis of secret key cryptosystems. In B. S. Kaliski Jr. (ed.), *Advances in Cryptology – CRYPTO ’97*, Volume 1294 of *Lecture Notes in Computer Science*, Springer-Verlag (1997) pp. 513–525.
6. D. Boneh, R. A. DeMillo and R. J. Lipton, On the importance of checking cryptographic protocols for faults. In W. Fumy (ed.), *Advances in Cryptology – EUROCRYPT ’97*, Volume 1233 of *Lecture Notes in Computer Science*, Springer-Verlag (1997) pp. 37–51.
7. D. Boneh, R. A. DeMillo and R. J. Lipton, On the importance of eliminating errors in cryptographic computations. *Journal of Cryptology*, Vol. 14, No. 2 (2001) pp. 101–119. An earlier version appears in [6].
8. E. De Win, S. Mister, B. Preneel and M. Wiener, On the performance of signature schemes based on elliptic curves. In J.-P. Buhler (ed.), *Algorithmic Number Theory Symposium*, Volume 1423 of *Lecture Notes in Computer Science*, Springer-Verlag (1998) pp. 252–266.
9. T. ElGamal, A public key cryptosystem and a signature scheme based on discrete logarithms. *IEEE Transactions on Information Theory*, Vol. IT-31, No. (4) (1985) pp. 469–472.
10. S. D. Galbraith, F. Hess and N. P. Smart, Extending the GHS Weil descent attack. In L. Knudsen (ed.), *Advances in Cryptology – EUROCRYPT 2002*, Volume 2332 of *Lecture Notes in Computer Science*, Springer-Verlag (2002) pp. 29–44.
11. P. Gaudry, F. Hess and N. P. Smart, Constructive and destructive facets of Weil descent on elliptic curves. *Journal of Cryptology*, Vol. 15, No. 1 (2002) pp. 19–46.

12. F. Hess, The GHS attack revisited. In E. Biham (ed.), *Advances in Cryptology – EUROCRYPT 2003*, Volume 2656 of *Lecture Notes in Computer Science*, Springer-Verlag (2003) 374–387.
13. M. Joye, J.-J. Quisquater, F. Bao and R. H. Deng, RSA-type signatures in the presence of transient faults. In M. Darnell, (ed.), *Cryptography and Coding*, Volume 1355 of *Lecture Notes in Computer Science*, Springer-Verlag (1997) pp. 155–160.
14. N. Koblitz, Elliptic curve cryptosystems. *Mathematics of Computation*, Vol. 48, No. 177 (1987) pp. 203–209.
15. P. Kocher, Timing attacks on implementations of Diffie-Hellman, RSA, DSS, and other systems. In N. Koblitz (ed.), *Advances in Cryptology – CRYPTO ’96*, Volume 1109 of *Lecture Notes in Computer Science*, Springer-Verlag (1996) pp. 104–113.
16. P. Kocher, J. Jaffe and B. Jun, Differential power analysis. In M. Wiener (ed.), *Advances in Cryptology – CRYPTO ’99*, Volume 1666 of *Lecture Notes in Computer Science*, Springer-Verlag (1999) pp. 388–397.
17. M. Maurer, A. J. Menezes and E. Teske, Analysis of the GHS Weil descent attack on the ECDLP over characteristic two finite fields of composite degree. In C. Pandu Rangan and C. Ding (ed.), *Progress in Cryptology – INDOCRYPT 2001*, Volume 2247 of *Lecture Notes in Computer Science*, Springer-Verlag (2001) pp. 195–213.
18. A. J. Menezes, *Elliptic Curve Public Key Cryptosystems*. Kluwer Academic Publishers (1993).
19. A. Menezes, T. Okamoto and S. Vanstone, Reducing elliptic curve logarithms to logarithms in a finite field. *IEEE Transactions on Information Theory*, Vol. 39 (1993) pp. 1639–1646.
20. A. J. Menezes and M. Qu, Analysis of the Weil descent attack of Gaudry, Hess and Smart. In D. Naccache (ed.), *Topics in Cryptology – CT-RSA 2001*, Volume 2020 of *Lecture Notes in Computer Science*, Springer (2001) pp. 308–318.
21. V. S. Miller, Use of elliptic curves in cryptography. In H. C. Williams (ed.), *Advances in Cryptology – CRYPTO ’85*, Volume 218 of *Lecture Notes in Computer Science*, Springer (1986) pp. 417–426.
22. J. M. Pollard, Monte Carlo methods for index computation (mod p). *Mathematics of Computation*, Vol. 32 (1978) pp. 918–924.
23. J. M. Pollard, Kangaroos, monopoly and discrete logarithms. *Journal of Cryptology*, Vol. 13, No. 4 (2000) pp. 437–447.
24. N. P. Smart, How secure are elliptic curves over composite extension fields? In B. Pfitzmann (ed.), *Advances in Cryptology – EUROCRYPT 2001*, Volume 2045 of *Lecture Notes in Computer Science*, Springer-Verlag (2001) pp. 30–39.
25. J. A. Solinas, Generalized Mersenne numbers. Technical Report CORR-99-39, Dept of C&O, University of Waterloo, Canada (1999).